

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



「 ANHANG: WEITERE SOFTWAREMASSE



Themen in dieser Vorlesung



Softwaremaße:

Zyklomatische Komplexität nach McCabe

Berechnung von M in zwei Alternativen

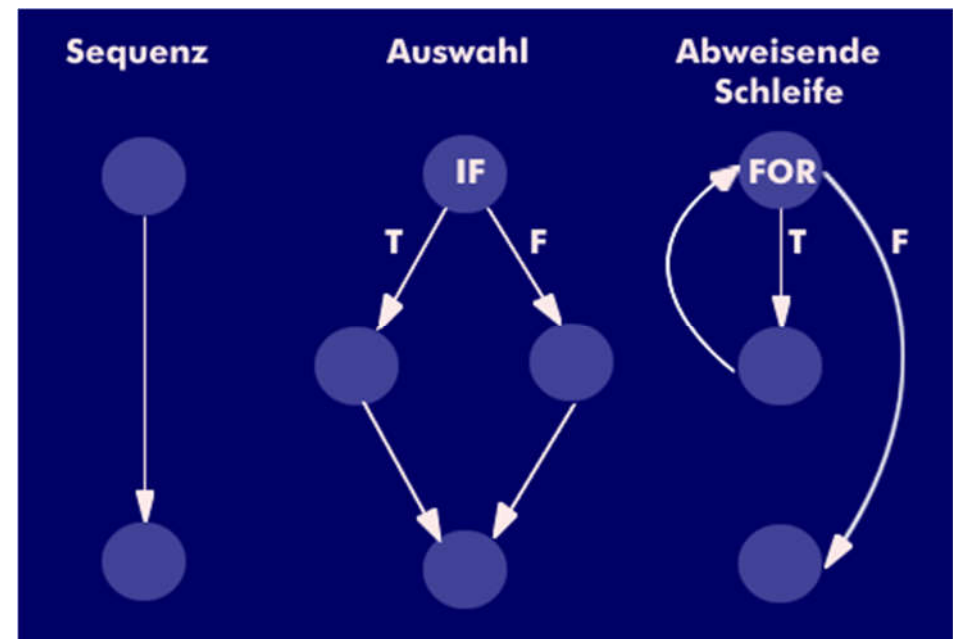
- $M = e - n + 2p$
 - e ist Anzahl der Kanten im Graph
 - n ist Anzahl der Knoten im Graph
 - p ist Anzahl der Knoten ohne ausgehende Kanten im Graphen
- $M = D + 1$
 - D ist Anzahl der Entscheidungspunkte

Softwaremaße:

Zyklomatische Komplexität nach McCabe

Beispiele für häufige Strukturen:

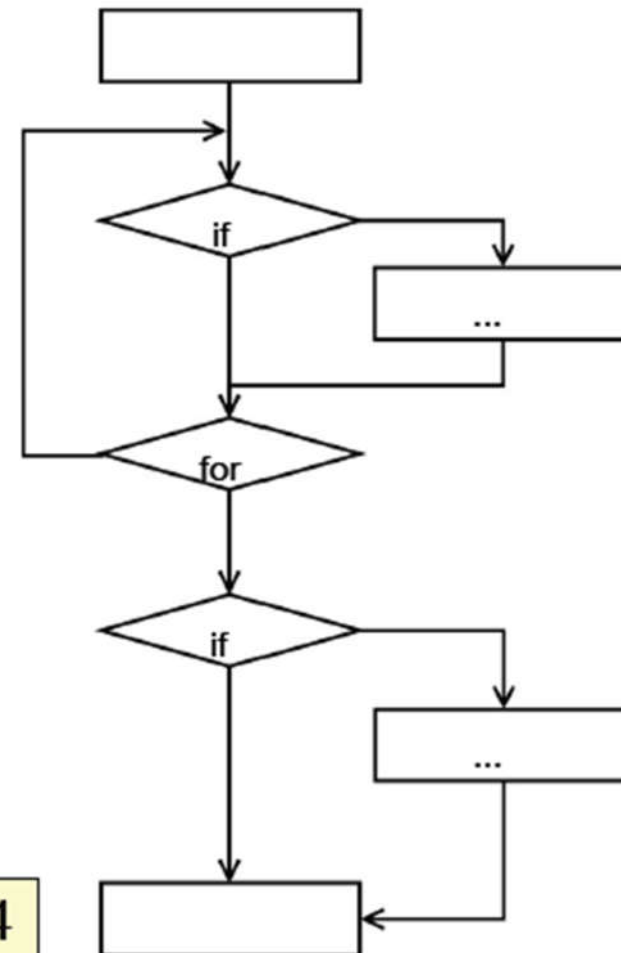
- Sequenz
 - $M = 1 - 2 + 2 = 1$
- Auswahl:
 - $M = 4 - 4 + 2 = 2$
- Abweisende Schleife:
 - $M = 3 - 3 + 2 = 2$



Beispiel: Bestimmung der zyklomatischen Komplexität nach McCabe

```
public void feed(Eatable[] eatable){  
    for (Eatable e : eatable){  
        if (hungry)  
            // iß etwas ...  
            ateCalories += e.getCalories();  
    }  
  
    if (ateCalories > 150){  
        // Sodbrennen  
        spitFire();  
    }  
}
```

$$c(F) = 9 - 7 + 2 = 4$$



Übung – Bestimmen Sie die zyklomatische Komplexität nach McCabe

```
1 package de.htwg.swen.metriken;
2
3 public class McCabe {
4
5     void beispiel(int n) {
6         int i, j, t;
7
8         int a[] = new int[n];
9         if (n < 2)
10             return;
11         for (i = 0; i < n - 1; i++) {
12             for (j = i + 1; j < n; j++) {
13                 if (a[i] > a[j]) {
14                     t = a[i];
15                     a[i] = a[j];
16                     a[j] = t;
17                 }
18             }
19         }
20     }
21 }
22 }
```

Softwaremaße: Diskussion

Zyklomatische Komplexität nach McCabe

- $M < 10$ wünschenswert
- Aussagekraft der Metrik in prozeduralen Programmen
- Hilfreich für Methoden
- Weniger relevant in strikt objektorientierten Programmen (Vererbung!)

Softwaremaße zur Unterstützung des Testens: Testabdeckung

- McCabe-Metrik / Cyclomatic Complexity
 - linear unabhängiger Pfade auf dem Kontrollflussgraphen
 - Obere Schranke für 100% Zweigabdeckung
- NPath Complexity
 - Anzahl möglicher Pfade durch eine Methode

Softwaremaße:

Weighted Methods per Class (WMC)

- Nach Chidamber / Kemerer: „Gewichtete Menge an Methoden“
- Berechnung: Summe der zyklomatischen Komplexität aller Methoden
- Überträgt zyklomatische Komplexität gewissermaßen auf Klassenebene
- Indikator für
 - Weiterentwicklungsaufwand
 - Wartungsaufwand
 - schwierige Wiederverwendung

Softwaremaße: Beispiel Weighted Methods per Class (WMC)

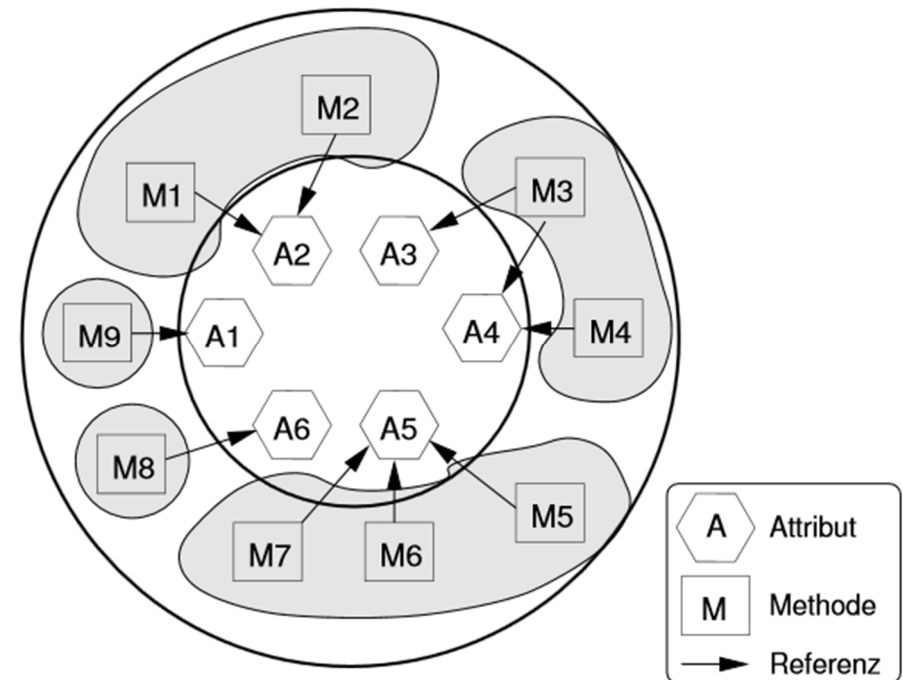
$$\text{WMC} = 5 + 4 = 9$$

```
1 package de.htwg.swen.metriken;
2
3 public class WmcMcCabe {
4
5     void beispiel(int n) {
6         int i, j, t;
7
8         int a[] = new int[n];
9         if (n < 2)
10             return;
11         for (i = 0; i < n - 1; i++) {
12             for (j = i + 1; j < n; j++) {
13                 if (a[i] > a[j]) {
14                     t = a[i];
15                     a[i] = a[j];
16                     a[j] = t;
17                 }
18             }
19         }
20     }
21
22     void beispiel2(int n) {
23         int i, j, t;
24
25         int a[] = new int[n];
26         for (i = 0; i < n - 1; i++) {
27             for (j = i + 1; j < n; j++) {
28                 if (a[i] > a[j]) {
29                     t = a[i];
30                     a[i] = a[j];
31                     a[j] = t;
32                 }
33             }
34         }
35     }
36 }
```

Softwaremaße:

Lack of cohesion on methods (LCOM1)

- Nach Chidamber / Kemerer: Messung von Kohäsionsmängeln der Methoden einer Klasse
- $LCOM1 = \max(P - Q, 0)$
 - P = Anzahl der Paare von Methoden einer Klasse, die disjunkte Instanzvariablen benutzen;
 - Q = Anzahl ... die mind. 1 Variable gemeinsam benutzen
- Indikator für
 - schlechtes Design
 - schlechte Klassenaufteilung
 - mögliches Trennen in mehrere Klassen



- 36 Methoden-Paare (8 mit M9 + 7 mit M8 + 6 mit M7 + ... + 1 mit M2)
- Q : 5 Paare (M7/M6, M7/M5, M6/M5, M3/M4, M1/M2)
- P : Alle anderen $\rightarrow 31$
- $LCOM1 = 31 - 5 = 26$

Softwaremaße: Response for Class (RFC)

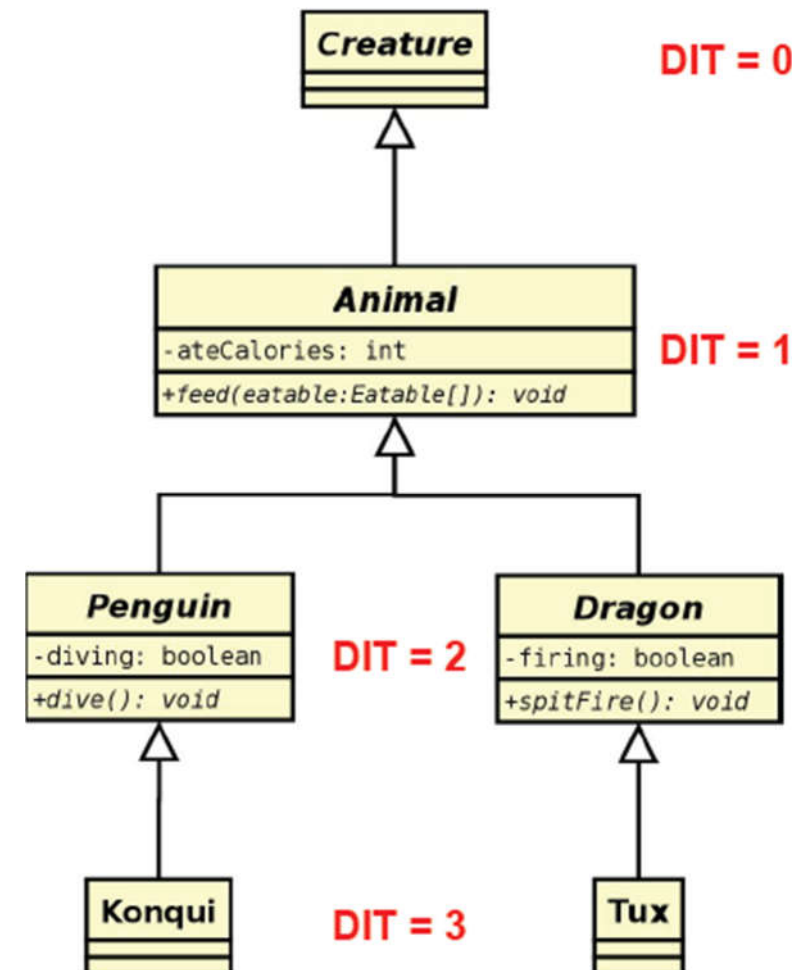
- Nach Chidamber / Kemerer
- **Berechnung:** (Anzahl aller verschiedener Methoden, die die Klasse aufruft) + (Anzahl aller Methoden der Klasse)
- Je größer RFC, desto komplexer ist Klasse und desto schwieriger sind Testen und finden von Fehlern

```
1  public class ClassA
2  {
3      private ClassB classB = new ClassB();    // call (constructor of class B) => +1
4      public void doSomething(){               // method declaration => +1
5          System.out.println ( "doSomething"); // call (System.out.println) => +1
6      }
7      public void doSomethingBasedOnClassB(){   // method declaration => +1
8          System.out.println (classB.toString()); // call (System.out.println) => 0 because already
9  counted on line 5 + call (toString) => +1
10     }
11 }
12
13 // default constructor of ClassA => +1
    // RFC = 6
```

Softwaremaße:

Tiefe der Vererbungshierarchie (DIT)

- Nach Chidamber / Kemerer: „Tiefe im Vererbungsbaum“
- Gibt Auskunft über Anzahl der Oberklassen
- Indikator für
 - Wiederverwendung
 - Strukturierung
 - Fehlerwahrscheinlichkeit
- Bessere Definition: DIT = Anzahl der Hierarchiestufen / Anzahl der Klassen





ANHANG: AUSGEWÄHLTE HEURISTIKEN ZUR AUFLÖSUNG VON STRUKTURPROBLEMEN



Themen in dieser Vorlesung



Heuristiken formalisieren und prüfbar (meßbar) machen

„Keine Flaschenhalsklassen“: Keine Klassen mit einer hohen Anzahl eingehender Aufrufe, die selber den Rest des Systems intensiv verwenden.

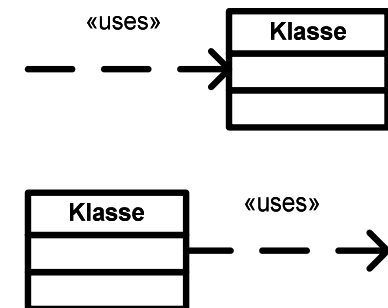
Wie läßt sich diese Heuristik prüfen?

Einsatz zweier Softwaremaße:

- Anzahl *eingehender* Aufrufe einer Klasse
- Anzahl *ausgehender* Aufrufe einer Klasse

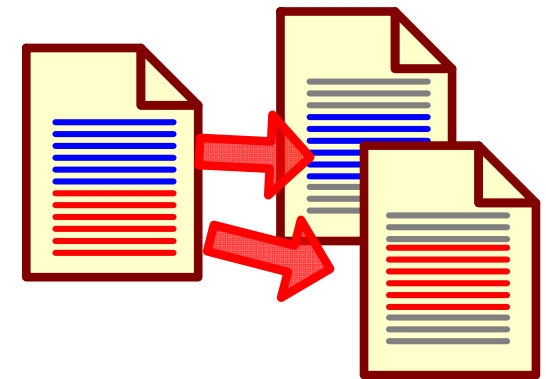
Festlegung von **Schwellwerten**: Etwa

- (*Eingehend* > 7) UND (*Ausgehend* > 7)
=> Flaschenhalsklasse



Spezielle Heuristik: Codeduplikation

- Duplizierter Code ist **wartungsunfreundlich**
 - Erhöht Systemgröße, Verständlichkeit leidet
 - Fehlerbeseitigungen meistens nur in einer Version
- **Lösung**, wenn Duplikatanzahl gering:
Ausfaktorisierung duplizierter Codeteile in Methode / Klasse
- **Erkennungstechnik:**
Zeilenbasiertes Pattern-Matching + „Clustering“
 - Sprachunabhängige Analyse
 - Aber: Kommentare zählen auch!

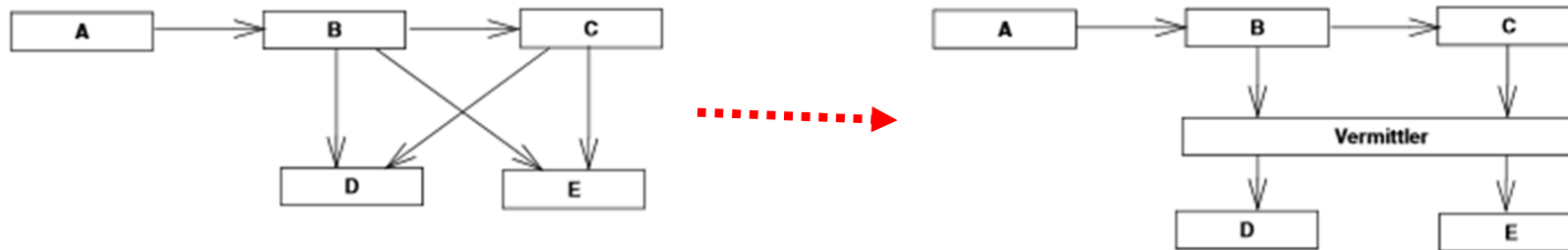


Heuristik I

Sind nur sehr wenige Klassen (Anhaltspunkt: bis zu drei) eng gekoppelt, versucht man, die Klassen zu einer einzigen Klasse zusammenzufassen

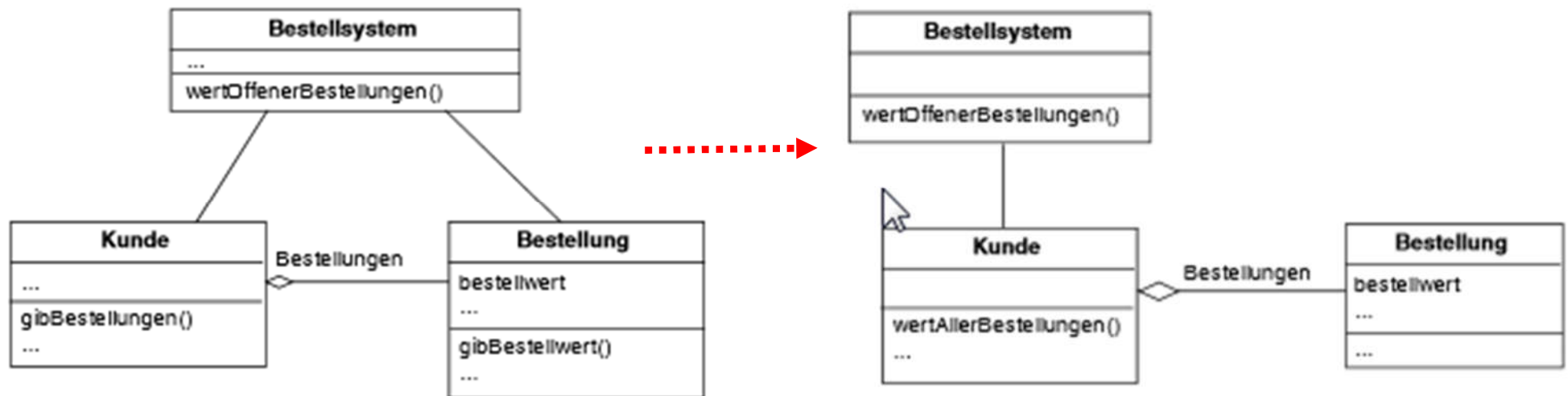
Heuristik II

Sind mehrere Klassen (mehr als drei) stark gekoppelt, versucht man, die Kopplung durch eine Vermittlerklasse abzuschwächen



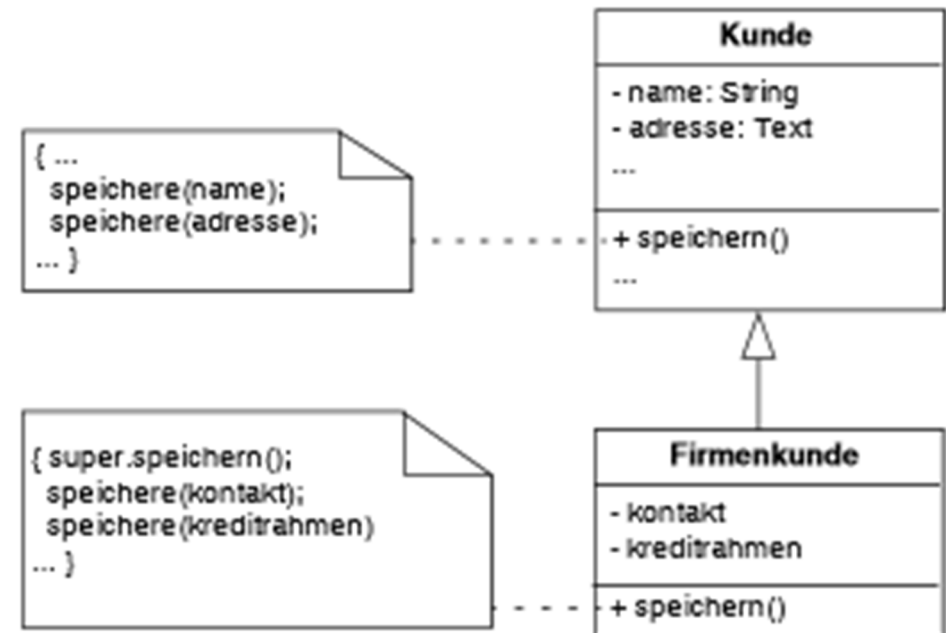
Heuristik III

Ist eine Funktionalität für die starke Kopplung verantwortlich, versucht man, sie in eine andere Klasse zu verschieben



Heuristik IV

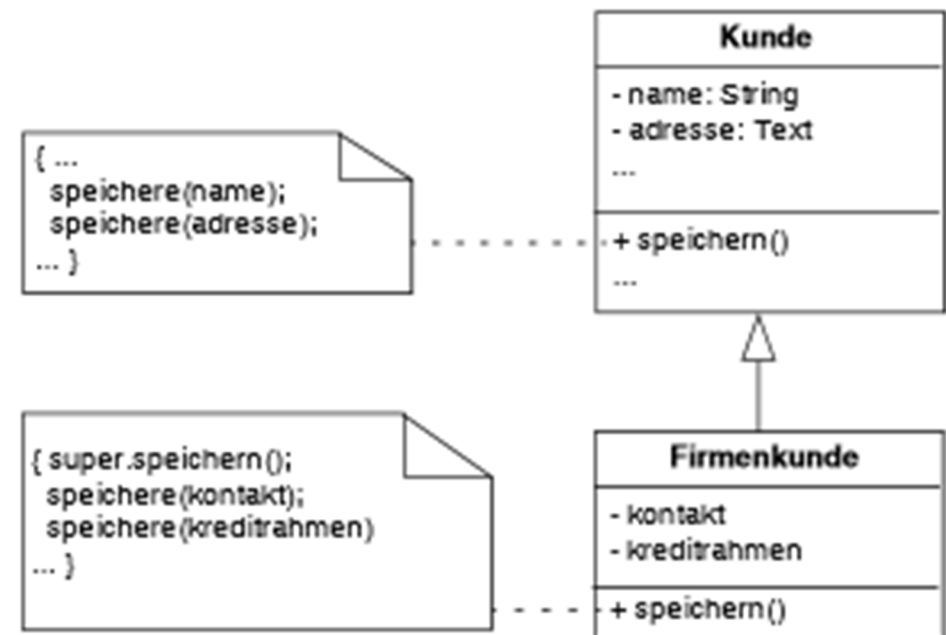
Sind innerhalb einer Generalisierungshierarchie *mehrere* Standardoperationen der Oberklasse für die starke Kopplung verantwortlich, versucht man, sie durch den Aufruf einer (komplexeren) Operation der Oberklasse zu vermeiden



Heuristik V

Lassen sich die Attribute und die Operationen einer Klasse in (weitgehend) disjunkte Teilmengen zerlegen, so dass jede Operationsmenge nur auf jeweils einer Attribut-menge arbeitet, dann ist die Aufteilung der Klasse zu empfehlen

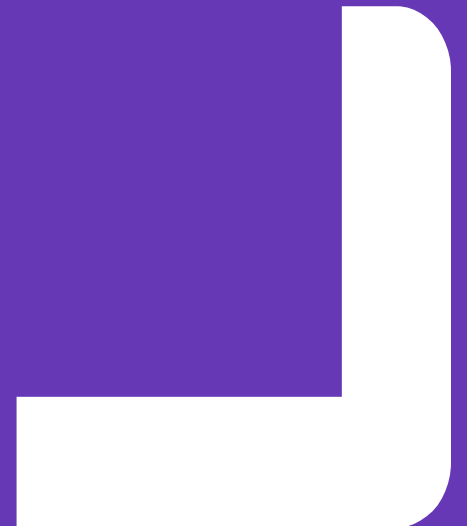
- Siehe auch LCOM-Maß





ANHANG: ERGÄNZUNG DES VORGESTELLTEN QUALITÄTSMODELLS VON LUDEWIG / LICHTER

Erläuterung der verwendeten Merkmale



Themen in dieser Vorlesung



Merkmale der Wartbarkeit

<i>Spezifikations-vollständigkeit:</i>	ist hoch, wenn die Spezifikation die tatsächlichen Anforderungen und nur diese vollständig angibt.
<i>Lokalität der Software:</i>	ist hoch, wenn Fernwirkungen in der Software (Wirkungen über die Grenzen der -Software-Komponenten hinweg) vermieden sind.
<i>Testbarkeit der Software:</i>	ist hoch, wenn die Programme unter definierten Bedingungen ausgeführt und die relevanten Resultate vollständig erfasst werden können. Die Ausführung ist damit reproduzierbar.
<i>Strukturiertheit:</i>	ist hoch, wenn die Software in logisch abgeschlossene Einheiten mit hohem Zusammenhalt und geringer Kopplung gegliedert ist.
<i>Simplizität:</i>	ist hoch, wenn in der Software nur wenige schwer verständliche Konstruktionen enthalten sind.
<i>Knappheit der Software:</i>	ist hoch, wenn ihr Umfang durch Vermeidung von Redundanz aller Art gering gehalten wurde.
<i>Lesbarkeit der Software:</i>	ist hoch, wenn ein (fremder) Leser in der Lage ist, mit minimalem Aufwand den Inhalt korrekt zu erfassen.
<i>Geräteunabhängigkeit:</i>	ist hoch, wenn Merkmale spezieller Geräte darin eine geringe Rolle spielen.
<i>Abgeschlossenheit:</i>	ist hoch, wenn die Software eine gut abgegrenzte Leistung erbringt und damit kaum Schnittstellen zu anderen Systemen hat.

Merkmale der Prozessqualität

Entwicklungseffizienz: ist hoch, wenn der Entwicklungsaufwand gering ist.

Entwicklungsgeschwindigkeit: ist hoch, wenn das Resultat nach kurzer Zeit zur Verfügung steht.

Termineinhaltung ist umso höher, je genauer der geplante Termin eingehalten wird.

Aufwandseinhaltung: ist umso höher, je genauer der geplante Aufwand eingehalten wird.

Prozesstransparenz: ist hoch, wenn der Bearbeitungsprozess wohldefiniert ist und gemäß der Definition abläuft.

Bausteingewinn: ist hoch, wenn viele wiederverwendbare Software-Komponenten entstehen oder verbessert werden.

Know-how-Gewinn: ist hoch, wenn die beteiligten Mitarbeiter viele neue Kenntnisse und Erfahrungen mit Anwendungen, Methoden und Werkzeugen erwerben.

Projektklima: ist gut, wenn die Mitarbeiter ihre Zusammenarbeit als angenehm empfinden und gern wieder ein ähnliches Projekt durchführen wollen.

Merkmale der Brauchbarkeit

<i>Korrektheit:</i>	ist hoch, wenn die Spezifikation zutreffend und die übrige Software korrekt in Bezug auf die Spezifikation ist.
<i>Ausfallsicherheit:</i>	ist hoch, wenn die Software nur selten die erwartete Funktion nicht erbringt.
<i>Genauigkeit:</i>	ist hoch, wenn die Resultate vom mathematisch korrekten Resultat nur wenig abweichen.
<i>Effizienz:</i>	ist hoch, wenn die Software kaum mehr Rechenzeit benötigt, als minimal erforderlich wäre.
<i>Sparsamkeit:</i>	ist hoch, wenn die Software kaum mehr Speicherplatz und andere Betriebsmittel benötigt, als minimal erforderlich wäre.
<i>Leistungs-vollständigkeit:</i>	ist hoch, wenn die Software alle geforderten Leistungen tatsächlich erbringt.
<i>Handbuch-vollständigkeit:</i>	ist hoch, wenn die Handbücher erschöpfend Auskunft auf alle sinnvollen Fragen des Benutzers geben.
<i>Konsistenz:</i>	ist hoch, wenn die Software sich gegen den Benutzer in ähnlichen Situationen ähnlich verhält. Das betrifft die Bedienung, Fehlermeldungen, auch Datenformate usw.
<i>Verständlichkeit:</i>	ist hoch, wenn der Benutzer rasch versteht, wie er mit der Software umgehen muss.
<i>Einfachheit:</i>	ist hoch, wenn die Software dem Benutzer konzeptionell einfach erscheint.